

ONBRD-04: Cloud & SaaS Integrations

Series: ONBRD — Dynatrace Onboarding | **Notebook:** 4 of 10 | **Created:** January 2026 | **Last Updated:** 09/18/2026

Extending Visibility Beyond OneAgent

While OneAgent provides deep application and infrastructure monitoring, many organizations need visibility into cloud services and SaaS platforms that can't run an agent. This notebook covers how to integrate AWS, Azure, GCP, and third-party SaaS tools into Dynatrace.

Table of Contents

1. [Integration Overview](#)
 2. [AWS Integration](#)
 3. [Azure Integration](#)
 4. [GCP Integration](#)
 5. [Extensions Framework](#)
 6. [Dynatrace Hub](#)
 7. [Common SaaS Integrations](#)
 8. [Verifying Integrations](#)
 9. [Next Steps](#)
-

Prerequisites

- Dynatrace environment with admin access

- **ActiveGate** deployed where required — Extensions and the classic GCP integration still need an AG; AWS (Clouds app GA) and Azure (Clouds app, SaaS 1.337+) use direct connections without AG
- Cloud provider admin access (for AWS/Azure/GCP)
- API credentials for SaaS platforms

1. Integration Overview

Dynatrace offers multiple integration methods depending on the data source:

Cloud & SaaS Integration Methods

Pick the method by data source — 2026-05 status

Clouds App

★ Modern · Direct connection

No ActiveGate
Agentless polling

Coverage
AWS — GA
Azure — SaaS 1.337+
GCP — Preview

→ Use for AWS / Azure

AG-Polling (Classic)

Requires ActiveGate

ActiveGate Polls
CloudWatch / Monitor / etc.

When to Use
GCP (classic integration)
AWS / Azure (no Clouds app)
Restricted networks

→ Use for GCP

Extensions

Extensions 2.0 · current framework

AG-Hosted
Custom polling logic

When to Use
Databases / SaaS APIs
SNMP / VMware / F5
On-host integrations

→ Custom data sources

OTel Direct

OTLP ingest

No ActiveGate
Direct OTLP

When to Use
OTel-instrumented apps + collectors
Polyglot / portable

→ Code-instrumented

Quick Decision Reference

AWS → Clouds app (GA), no AG required

Azure → Clouds app (SaaS 1.337+), no AG required

GCP → Classic AG-polling; Clouds app in Preview

AWS Lambda → Clouds app + DT_TAGS env var (sprint-1.337)

DB / SaaS / SNMP → Extensions 2.0 on AG

OTel app → Direct OTLP ingest, no AG

See CLOUD series for per-provider deep dives · AUTOM series for extension automation

Method	Use Case	Requires ActiveGate?
Clouds App (recommended where supported)	AWS (GA), Azure (SaaS 1.337+); GCP in Preview — verify it has reached your tenant	No (direct connection)
Cloud Integrations (Classic / AG-polling)	GCP (classic integration — the working path until the Preview reaches you); AWS / Azure where Clouds app is not used	Yes
Extensions 2.0 (current framework)	Custom data sources, SaaS APIs, on-host integrations	Yes (AG-hosted)
OpenTelemetry	OTel-instrumented apps	No (direct OTLP ingest)

Method	Use Case	Requires ActiveGate?
Log Ingest	External log sources	Optional (AG can route)
Metrics Ingest	Custom metrics via API	No

Decision Rule (Quick Reference)

Source	Default Method (09/2026)
AWS	Clouds app (GA) — direct connection
Azure	Clouds app — direct connection (SaaS 1.337+); <i>"no need to deploy ActiveGate compute resources for metric polling"</i> (Azure Cloud Platform Monitoring (DT docs))
GCP	Classic AG-based integration until the Clouds-app Preview reaches your tenant; then evaluate the Clouds-app connection
AWS Lambda	Clouds app + DT_TAGS env var (sprint-1.337) for tag propagation
Custom DB / SaaS / SNMP	Extensions 2.0 on AG
OTel-instrumented app	OTLP ingest direct to Dynatrace

Why Integrate Before OneAgent?

Benefit	Description
Infrastructure context	See cloud resources before deploying agents
Dependency mapping	Understand managed services (RDS, Lambda, etc.)
Cost visibility	Cloud cost data available immediately
Complete topology	Smartscape includes cloud services from day one

Where to go deeper: the **CLOUD series** (9 notebooks) covers per-provider deep dives (AWS, Azure, GCP). The **AUTOM series** covers GitOps/Terraform automation

for extension deployment at scale.

2. AWS Integration

The AWS integration pulls metrics from CloudWatch and discovers AWS resources.

Supported Services

Category	Services
Compute	EC2, Lambda, ECS, EKS, Fargate
Database	RDS, DynamoDB, ElastiCache, DocumentDB
Storage	S3, EBS, EFS
Networking	ELB, ALB, NLB, API Gateway, CloudFront
Messaging	SQS, SNS, Kinesis, MSK
Other	Step Functions, Secrets Manager, and more

Setup Methods

Method	Best For	Complexity
IAM Role (recommended)	Production, multi-account	Medium
Access Key	Quick testing	Low
CloudFormation	Automated setup	Low

IAM Role Setup

1. Create an IAM role with CloudWatch read permissions
2. Add trust relationship for Dynatrace's AWS account
3. Configure in Dynatrace: Settings → Cloud and virtualization → AWS

Required IAM Policy:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "cloudwatch:GetMetricData",
        "cloudwatch:GetMetricStatistics",
        "cloudwatch:ListMetrics",
        "tag:GetResources",
        "tag:GetTagKeys",
        "tag:GetTagValues",
        "ec2:DescribeInstances",
        "ec2:DescribeVolumes",
        "rds:DescribeDBInstances",
        "lambda:ListFunctions",
        "lambda:GetFunction"
      ],
      "Resource": "*"
    }
  ]
}
```

Configuration Location

Path: Settings → Cloud and virtualization → AWS

Setting	Recommendation
Polling interval	5 minutes (default)
Services to monitor	Start with core services, expand as needed
Regions	Only regions where you have resources
Resource tags	Use tags to filter monitored resources

3. Azure Integration

The Azure integration uses Azure Monitor to collect metrics and discover resources.

Supported Services

Category	Services
Compute	Virtual Machines, App Service, Functions, AKS
Database	SQL Database, Cosmos DB, Redis Cache
Storage	Blob, Files, Queues, Tables
Networking	Load Balancer, Application Gateway, VNet
Messaging	Service Bus, Event Hubs, Event Grid

Setup via App Registration

1. Create an App Registration in Azure AD
2. Grant **Reader** role on subscriptions to monitor
3. Create a client secret
4. Configure in Dynatrace with:
 - Tenant ID
 - Client ID
 - Client Secret
 - Subscription IDs

Configuration Location: Settings → Cloud and virtualization → Azure

4. GCP Integration

The GCP integration uses Cloud Monitoring (formerly Stackdriver) APIs.

Status note (09/2026): Dynatrace's GCP setup page now lists **GCP Cloud Platform Monitoring (Preview)**: *"Connect your Google Cloud accounts to Dynatrace and manage the newly created GCP connections entirely from Clouds."* Preview features reach tenants on their own schedule — verify it has reached yours before planning around it. Until it does, the classic integration below, polled through an **ActiveGate**, remains the working path. Whether the Preview connection needs an ActiveGate is

not stated here — check [Set up Dynatrace on Google Cloud \(DT docs\)](#). Review the **CLOUD series** for per-provider detail.

Supported Services

Category	Services
Compute	Compute Engine, GKE, Cloud Run, Cloud Functions
Database	Cloud SQL, Cloud Spanner, Firestore, Bigtable
Storage	Cloud Storage
Networking	Load Balancing, Cloud CDN
Messaging	Pub/Sub

Setup via Service Account

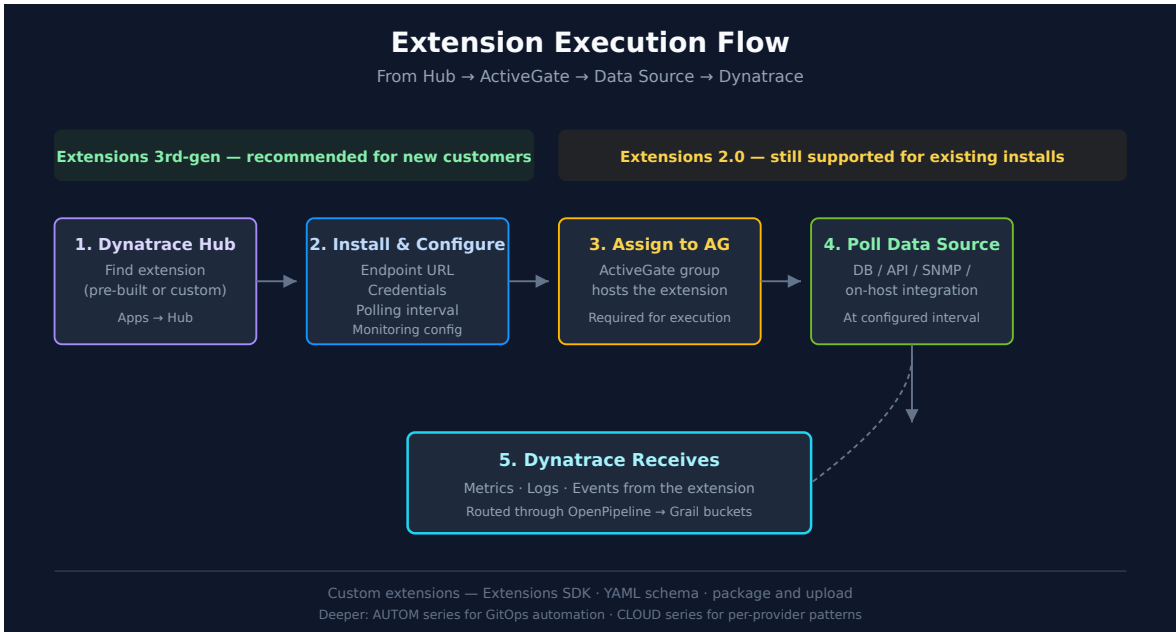
1. Create a Service Account in GCP
2. Grant **Monitoring Viewer** role
3. Generate and download JSON key
4. Configure in Dynatrace

Configuration Location: Settings → Cloud and virtualization → Google Cloud Platform

5. Extensions Framework

Extensions are Dynatrace's framework for integrating any data source — databases, SaaS platforms, network devices, on-host integrations, and more.

Recommendation for new customers: build on **Extensions 2.0** — the current extensions framework. Extensions Framework 1.0 reached end of support on 2025-03-31 (Python EF1.0: 2024-10-31); JMX and PMI EF1.0 are deprecated but supported past that date on request. Recent Dynatrace doc pages say simply "Extensions" and mean Extensions 2.0.



How Extensions Work

Component	Role
Extension Package	Code + metadata defining what to collect
ActiveGate	Hosts and executes the extension; polls data sources
Monitoring Configuration	Instance-specific settings (endpoints, credentials)
Dynatrace	Receives metrics, logs, events from the extension

Extension Categories

Category	Examples
Database	Oracle, SQL Server, PostgreSQL, MongoDB, Redis, Cassandra, MariaDB, DB2, HANA
On-host integrations	NGINX, HAProxy, Kafka, RabbitMQ, Elasticsearch, Memcached, Couchbase, Consul, Apache, etcd, Varnish, Zookeeper
Infrastructure	VMware, SNMP devices, F5, NetApp
SaaS	Salesforce, ServiceNow, Jira, Confluent
Custom	Any REST API, custom protocols

Installing an Extension

1. **Find the extension** in Dynatrace Hub
2. **Install** to your environment
3. **Configure** with endpoint and credentials
4. **Assign** to an ActiveGate group
5. **Verify** data is flowing

Extension Configuration Example

```
# Example: Generic REST API extension configuration
enabled: true
description: "My SaaS API"
endpoints:
  - url: "https://api.example.com/metrics"
    authentication:
      type: "bearer"
      token: "${SAAS_API_TOKEN}"
      polling_interval: 60
activeGate:
  group: "saas-integrations"
```

Creating Custom Extensions

For SaaS platforms without a pre-built extension, you can build a custom extension:

1. Use the Extensions 2.0 SDK
2. Define metrics schema in YAML
3. Write the polling logic
4. Package and upload to Dynatrace

SDK Documentation: [Extensions Development](#)

6. Dynatrace Hub

The Dynatrace Hub is your marketplace for extensions, integrations, and apps.

Accessing the Hub

Path: Apps → Dynatrace Hub

Or use quick search: **Cmd+K** → "Hub"

Hub Categories

Category	Examples
Cloud	AWS, Azure, GCP, Kubernetes
Database	Oracle, SQL Server, PostgreSQL
Infrastructure	VMware, SNMP, NetApp
Messaging	Kafka, RabbitMQ, IBM MQ
Observability	OpenTelemetry, Prometheus
Security	Snyk, SonarQube
ITSM	ServiceNow, Jira, PagerDuty

Installing from Hub

1. Search for the integration you need
2. Click **Install** or **Add to environment**
3. Follow the configuration wizard
4. Verify data appears in Dynatrace

7. Common SaaS Integrations

Messaging Platforms

Platform	Integration Method	Key Metrics
Confluent Cloud	Extensions 2.0	Consumer lag, throughput, partitions
Amazon MSK	AWS Integration	Broker metrics, topic metrics

Platform	Integration Method	Key Metrics
RabbitMQ	Extensions 2.0	Queue depth, message rates

CRM & Business Apps

Platform	Integration Method	Key Metrics
Salesforce	Extensions 2.0 / API	API calls, response times, limits
ServiceNow	Extensions 2.0	Incident metrics, workflow times

DevOps Tools

Platform	Integration Method	Key Metrics
GitHub	Webhooks	Deployment events, commit info
GitLab	Webhooks	Pipeline events, deployments
ArgoCD	Extensions 2.0	Sync status, app health

Database-as-a-Service

Platform	Integration Method	Key Metrics
MongoDB Atlas	Extensions 2.0	Connections, ops/sec, replication lag
AWS DocumentDB	AWS Integration	CloudWatch metrics
Snowflake	Guided configuration (SaaS 1.344+) / Extensions 2.0	Query performance, warehouse usage

Snowflake monitoring is changing (SaaS 1.344, July 2026). SaaS 1.344 was published 07/27/2026 with a **staged tenant rollout** beginning 07/29/2026, and it replaces the legacy Snowflake extension with a **guided configuration wizard** — verify the wizard has reached your tenant before planning around it. Until it arrives, **Extensions 2.0 remains the working method**, and existing extension-based

Snowflake configurations keep running; there is no need to tear one down ahead of the wizard.

Do not confuse this with the Snowflake *data connector*. The connector installed from the Hub and configured under **Settings > Connections** powers Snowflake *workflow actions* (Execute Statement, Store Statement Result) — see the WFLOW series for that path. It is a different surface, serves a different purpose, and is unaffected by this change.

8. Verifying Integrations

After configuring integrations, verify data is flowing into Dynatrace.

```
// Check for AWS Lambda functions (modern Smartscape topology query)
smartscapeNodes "AWS_LAMBDA_FUNCTION"
| fields name, id
| limit 20

// Legacy alternative (deprecated dt.entity.* – still works on hybrid
tenants):
// fetch dt.entity.aws_lambda_function
// | fields entity.name, id
// | limit 20
```

```
// Check for Azure VMs (modern Smartscape topology query)
// Azure node types follow AZURE_<RESOURCE_PROVIDER>_<TYPE> -
"AZURE_VM" is not a node type and returns nothing.
smartscapeNodes "AZURE_MICROSOFT_COMPUTE_VIRTUALMACHINES"
| fields name, id
| limit 20

// Legacy alternative (deprecated dt.entity.* – still works on hybrid
tenants):
// fetch dt.entity.azure_vm
// | fields entity.name, id
// | limit 20
```

```
// Check for cloud-sourced metrics
//
// The `dt.cloud.aws.*` metric namespace DOES NOT EXIST in any spelling
// (verified 08/12/2026).
// AWS CloudWatch metrics are published as
cloud.aws.<service>.<CloudWatchName>.By.<Dimension> -
// the CloudWatch name keeps its PascalCase and the split dimension is part of
// the key. A
// timeseries against a missing key returns an EMPTY result rather than an
// error, so this cell
// silently drew nothing. Enumerate with:
// metrics | filter startsWith(metric.key, "cloud.aws") | fields metric.key
// | sort metric.key asc
timeseries avgDuration = avg(cloud.aws.lambda.Duration.By.FunctionName),
from:-24h, by:{FunctionName}
| fieldsAdd avg_duration_ms = arrayAvg(avgDuration)
| fields FunctionName, avg_duration_ms
| sort avg_duration_ms desc
| limit 10

// Or browse available cloud metrics in the Dynatrace Metrics browser:
// Observe & Explore → Metrics → search for "cloud" or "aws" / "azure" /
"gcp"
```

```
// Check Extensions 2.0 status
// Note: "dt.entity.extensions:extension" is not a valid entity type for DQL
// fetch.
// Use the Extensions API v2 instead:
// GET /api/v2/extensions
// Or check installed extensions in the Dynatrace Hub:
// Apps → Dynatrace Hub → Installed
// To list extension-reported entities, query a known extension entity type:
fetch dt.entity.host
| fieldsKeep entity.name, id
| limit 10

// Smartscape equivalent (dt.entity.* is deprecated but still functional):
// smartscapeNodes "HOST"
// | fieldsKeep name, id
// | limit 10
// Caveat: Smartscape reflects CURRENT live topology and can report fewer
// entities
// than the classic entity store; for a pre-migration discovery inventory keep
// the
// classic query above.
// Field maps: entity.name -> name.
```

Troubleshooting Integration Issues

Issue	Common Cause	Solution
No data appearing	Credentials invalid	Verify IAM role/service account
Partial data	Permissions incomplete	Check required permissions list
Delayed data	Polling interval	CloudWatch has 5-min delay by design
Extension not running	ActiveGate issue	Check AG logs, verify assignment
Metrics but no entities	Tag configuration	Ensure resources are tagged correctly

9. Next Steps

With cloud and SaaS integrations configured:

1. **ONBRD-05: Deploying OneAgent** — Add deep application monitoring
2. **ONBRD-06: Organizing Your Environment** — Tag and segment integrated resources; design `dt.security_context`
3. Explore the topology in Smartscape to see cloud services
4. Create dashboards combining cloud metrics with application data

Where to Go Deeper

- **CLOUD series** (9 notebooks) — Per-provider integration deep dives (AWS, Azure, GCP)
- **AUTOM series** (14 notebooks) — GitOps / Terraform / Monaco automation for extension deployment
- **OPLOGS / OPMIG / OPIPE** — OpenPipeline routing for ingested cloud and SaaS data
- **OTEL series** — OpenTelemetry as the alternative ingest path

Integration Checklist

- [] AWS / Azure / GCP integration configured (Clouds app for AWS/Azure; classic AG polling for GCP until the Clouds-app Preview reaches your tenant)
- [] Required cloud permissions granted
- [] ActiveGate assigned for Extensions

- [] Extensions 2.0 evaluated for custom integrations; any remaining EF1.0 extensions identified for migration
 - [] Key SaaS platforms identified for integration
 - [] Extensions installed from Hub
 - [] Data verified flowing into Dynatrace
-

Summary

In this notebook, you learned:

- The integration method matrix (Clouds app / AG polling / Extensions / OTel / Ingest APIs)
 - Per-provider status (AWS GA, Azure direct connection since SaaS 1.337, GCP Clouds app in Preview)
 - AWS, Azure, and GCP setup paths
 - Extensions 2.0 as the current extensions framework
 - Dynatrace Hub for discovery
 - Common SaaS integration patterns
 - DQL queries to verify integrations are landing in topology
-

References

- [Cloud Platforms](#)
- [Clouds App](#)
- [AWS Monitoring](#)
- [Azure Monitoring](#)
- [Azure Cloud Platform Monitoring \(DT docs\)](#) — the no-ActiveGate quote above
- [What's new in Dynatrace SaaS 1.337 \(DT docs\)](#)
- [Set up Dynatrace on Google Cloud \(DT docs\)](#)
- [Extensions Framework](#)
- [Dynatrace Hub](#)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.